



BITS Pilani Digital Design

BITS Pilani
Pilani Campus

Prof. GSS Chalapathi, Prof. Saurabh Gandhi (EEE department)

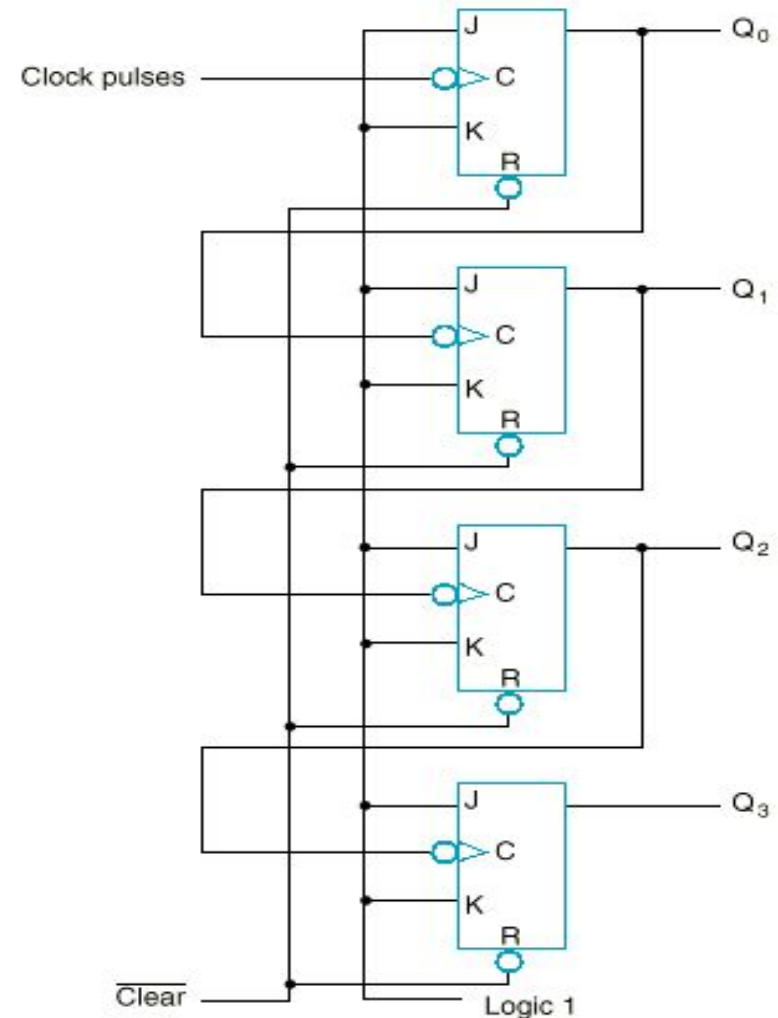
Counters

- A counter is a register that goes through a predetermined sequence of states upon the application of clock pulses.
- Categorized as:
 - Ripple counters
The FF output transition serves as a source for triggering other FFs.
NO COMMON CLOCK.
 - Synchronous counters
All FFs receive the common clock pulse, and the change of state is determined based on the present state

4-bit binary up counter

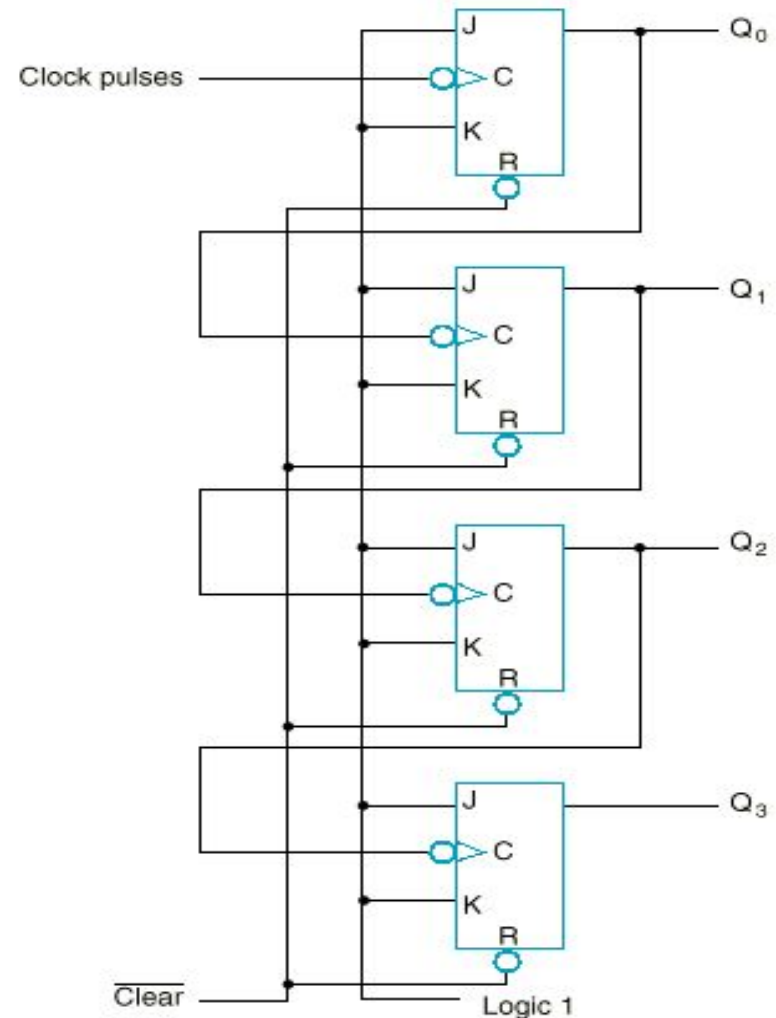
- Less significant bit serves as the clock for the next significant bit (negative edge clock)

(a) JK Flip-Flop			
J	K	$Q(t+1)$	Operation
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$\overline{Q}(t)$	Complement



4-bit binary up counter

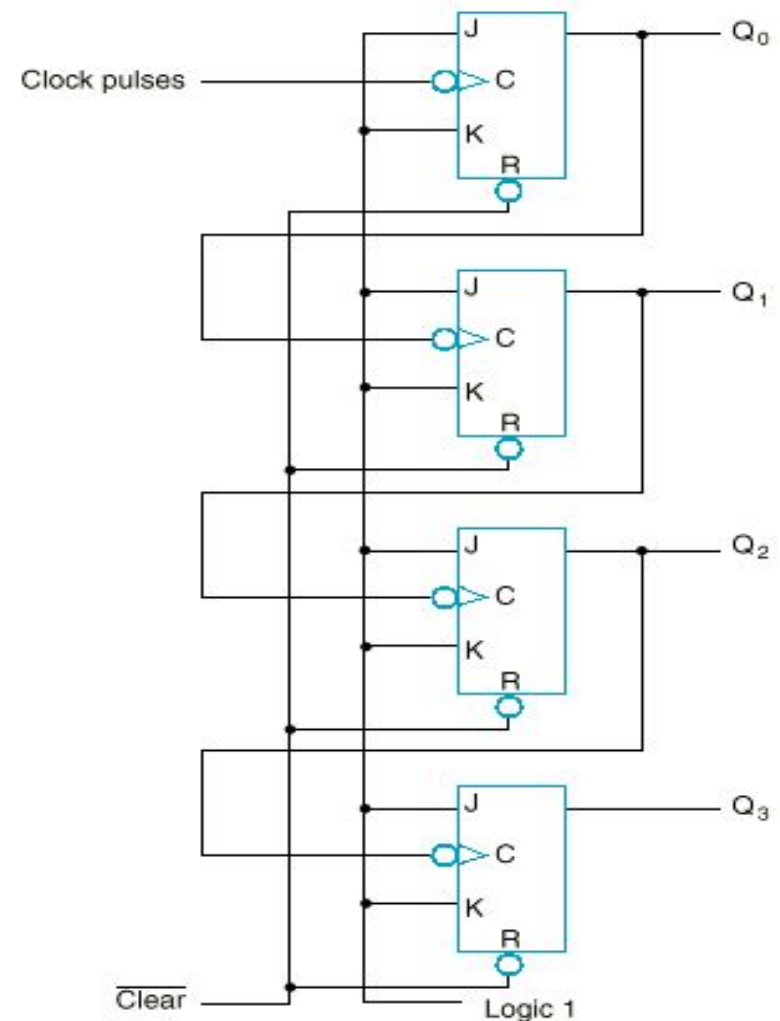
- The output of each FF is connected to the C input of the next FF in sequence.
- The FF holding the least significant bit receives the incoming clock pulses.
- The J and K inputs of all FFs are connected to a permanent logic 1.
- The bubble next to the C label indicates that the FFs respond to the negative-going transition of the input.



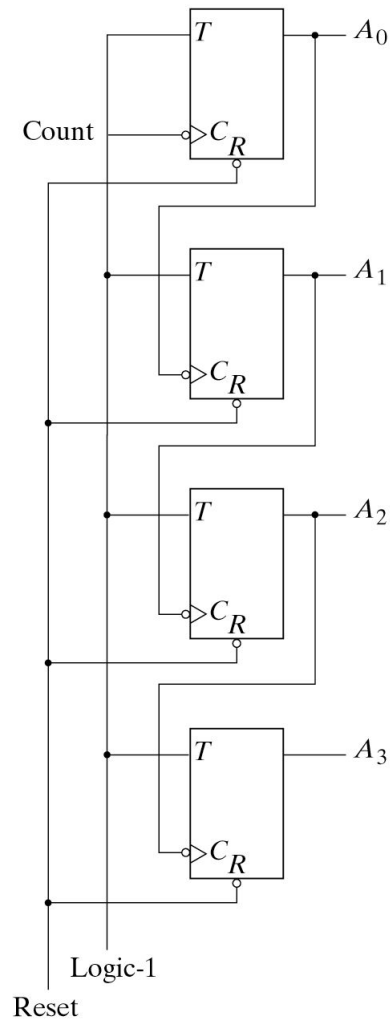
4-bit binary up counter



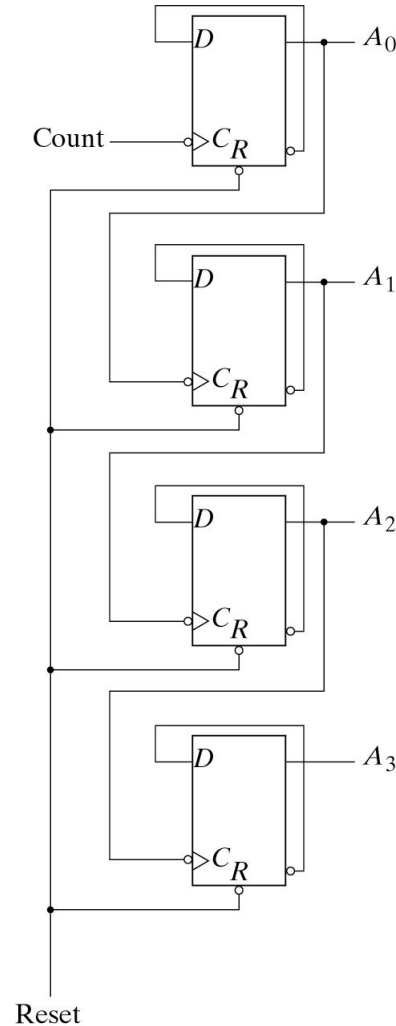
Upward Counting Sequence			
Q_3	Q_2	Q_1	Q_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1



4-bit ripple counters (other designs)



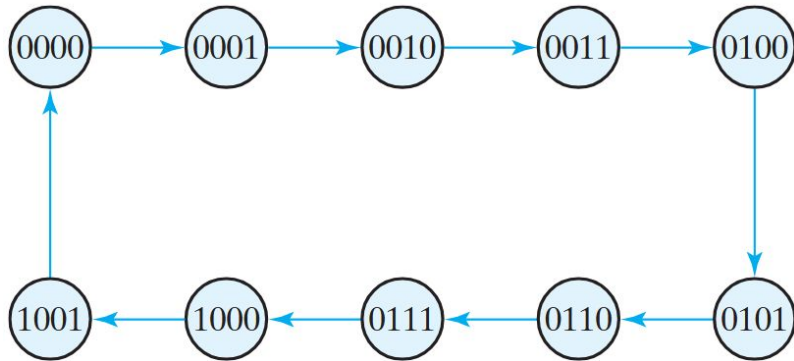
(a) With T flip-flops



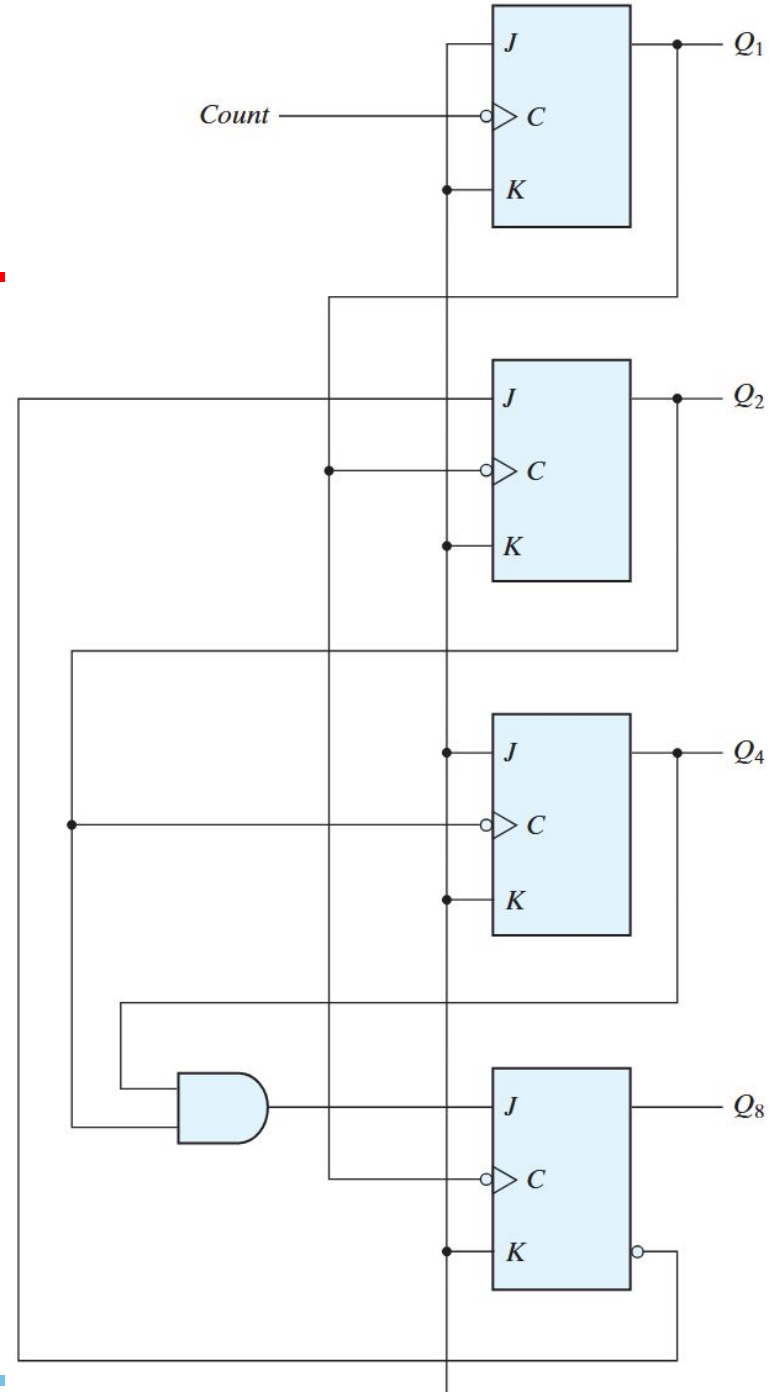
(b) With D flip-flops

4-bit binary down counter

- Use direct set (S) instead of direct reset (R) to start at 1111 instead of 0000
- Design the rest on your own...

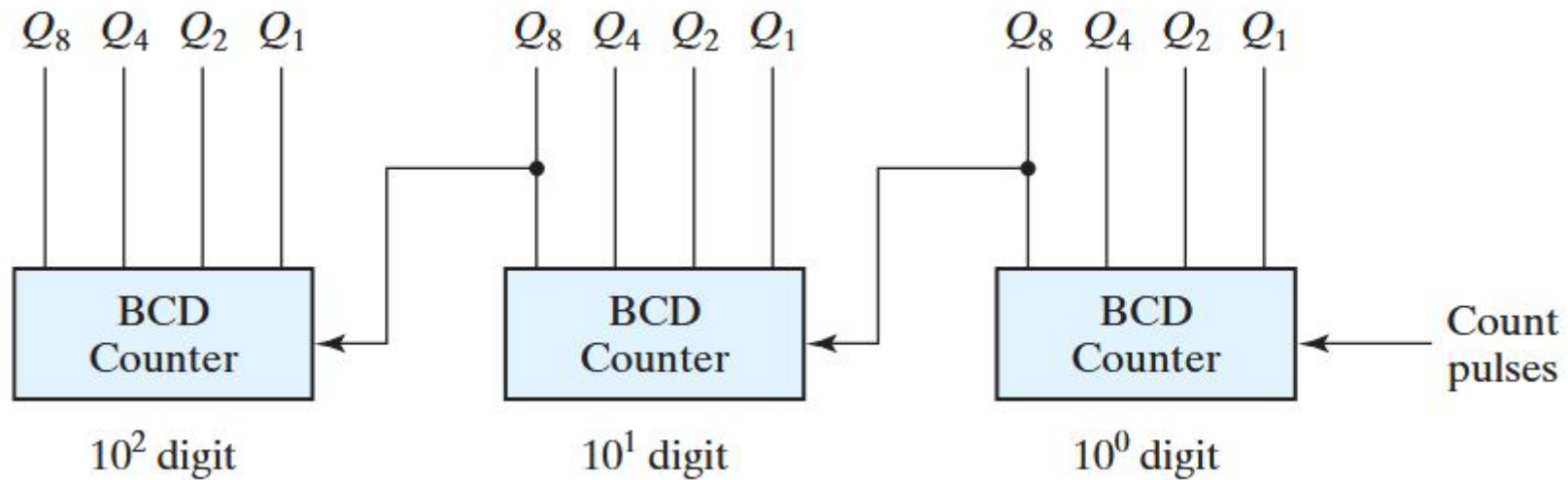


- Q1 complemented on every clock edge
- Q2 complemented if $Q8 = 0$, Q1 goes from 1 to 0.
- Q2 cleared if $Q8 = 1$ and Q1 goes from 1 to 0
- Q4 complemented when Q2 goes from 1 to 0
- Q8 complemented when $Q4Q2 = 11$, Q1 goes from 1 to 0.
- Q8 cleared if either Q4 or Q2 is 0 and Q1 goes from 1 to 0



Logic 1

The 'ripple' in BCD ripple counters



Synchronous binary counters

- Same design procedure as other sequential circuits
- Inputs: clock and control signals (EN, Load etc)
- Outputs: FF states
- Most efficient implementations use T or JK FFs

JK synchronous binary counter



TABLE 4-10
Flip-Flop Excitation Tables

(a) JK Flip-Flop

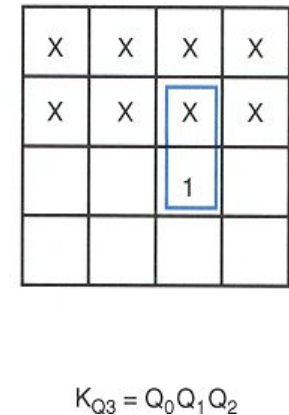
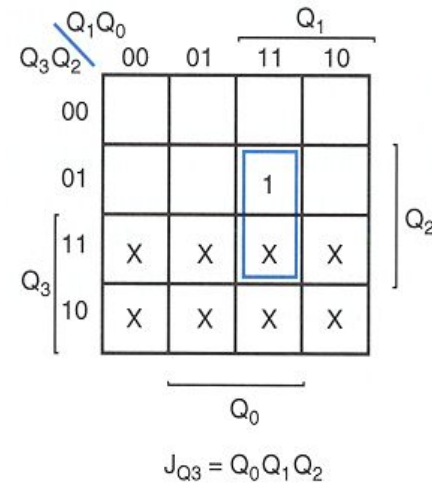
$Q(t)$	$Q(t+1)$	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Present state				Next state				Flip-flop inputs							
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0	J_{Q3}	K_{Q3}	J_{Q2}	K_{Q2}	J_{Q1}	K_{Q1}	J_{Q0}	K_{Q0}
0	0	0	0	0	0	0	1	0	X	0	X	0	X	1	X
0	0	0	1	0	0	1	0	0	X	0	X	1	X	X	1
0	0	1	0	0	0	1	1	0	X	0	X	X	0	1	X
0	0	1	1	0	1	0	0	0	X	1	X	X	1	X	1
0	1	0	0	0	1	0	1	0	X	X	0	0	X	1	X
0	1	0	1	0	1	1	0	0	X	X	0	1	X	X	1
0	1	1	0	0	1	1	1	0	X	X	0	X	0	1	X
0	1	1	1	1	0	0	0	1	X	X	1	X	1	X	1
1	0	0	0	1	0	0	1	X	0	0	X	0	X	1	X
1	0	0	1	1	0	1	0	X	0	0	X	1	X	X	1
1	0	1	0	1	0	1	1	X	0	0	X	X	0	1	X
1	0	1	1	1	1	0	0	X	0	1	X	X	1	X	1
1	1	0	0	1	1	0	1	X	0	X	0	0	X	1	X
1	1	0	1	1	1	1	0	X	0	X	0	1	X	X	1
1	1	1	0	1	1	1	1	X	0	X	0	X	0	1	X
1	1	1	1	0	0	0	0	X	1	X	1	X	1	X	1

JK synchronous binary counter



Present state				Next state				J_{Q3}	K_{Q3}
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0		
0	0	0	0	0	0	0	1	0	X
0	0	0	1	0	0	1	0	0	X
0	0	1	0	0	0	1	1	0	X
0	0	1	1	0	1	0	0	0	X
0	1	0	0	0	1	0	1	0	X
0	1	0	1	0	1	1	0	0	X
0	1	1	0	0	1	1	1	0	X
0	1	1	1	1	0	0	0	1	X
1	0	0	0	1	0	0	1	X	0
1	0	0	1	1	0	1	0	X	0
1	0	1	0	1	0	1	1	X	0
1	0	1	1	1	1	0	0	X	0
1	1	0	0	1	1	0	1	X	0
1	1	0	1	1	1	1	0	X	0
1	1	1	0	1	1	1	1	X	0
1	1	1	1	0	0	0	0	X	1



JK synchronous binary counter



Present state				Next state				J_{Q2}	K_{Q2}
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0		
0	0	0	0	0	0	0	1	0	X
0	0	0	1	0	0	1	0	0	X
0	0	1	0	0	0	1	1	0	X
0	0	1	1	0	1	0	0	1	X
0	1	0	0	0	1	0	1	X	0
0	1	0	1	0	1	1	0	X	0
0	1	1	0	0	1	1	1	X	0
0	1	1	1	1	0	0	0	X	1
1	0	0	0	1	0	0	1	0	X
1	0	0	1	1	0	1	0	0	X
1	0	1	0	1	0	1	1	0	X
1	0	1	1	1	1	0	0	1	X
1	1	0	0	1	1	0	1	X	0
1	1	0	1	1	1	1	0	X	0
1	1	1	0	1	1	1	1	X	0
1	1	1	1	0	0	0	0	X	1

		1	
X	X	X	X
X	X	X	X
		1	

$$J_{Q2} = Q_0 Q_1$$

X	X	X	X
		1	
		1	
X	X	X	X

$$K_{Q2} = Q_0 Q_1$$

JK synchronous binary counter



Present state				Next state				J_{Q1}	K_{Q1}
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0		
0	0	0	0	0	0	0	1	0	X
0	0	0	1	0	0	1	0	1	X
0	0	1	0	0	0	1	1	X	0
0	0	1	1	0	1	0	0	X	1
0	1	0	0	0	1	0	1	0	X
0	1	0	1	0	1	1	0	1	X
0	1	1	0	0	1	1	1	X	0
0	1	1	1	1	0	0	0	X	1
1	0	0	0	1	0	0	1	0	X
1	0	0	1	1	0	1	0	1	X
1	0	1	0	1	0	1	1	X	0
1	0	1	1	1	1	0	0	X	1
1	1	0	0	1	1	0	1	0	X
1	1	0	1	1	1	1	0	1	X
1	1	1	0	1	1	1	1	X	0
1	1	1	1	0	0	0	0	X	1

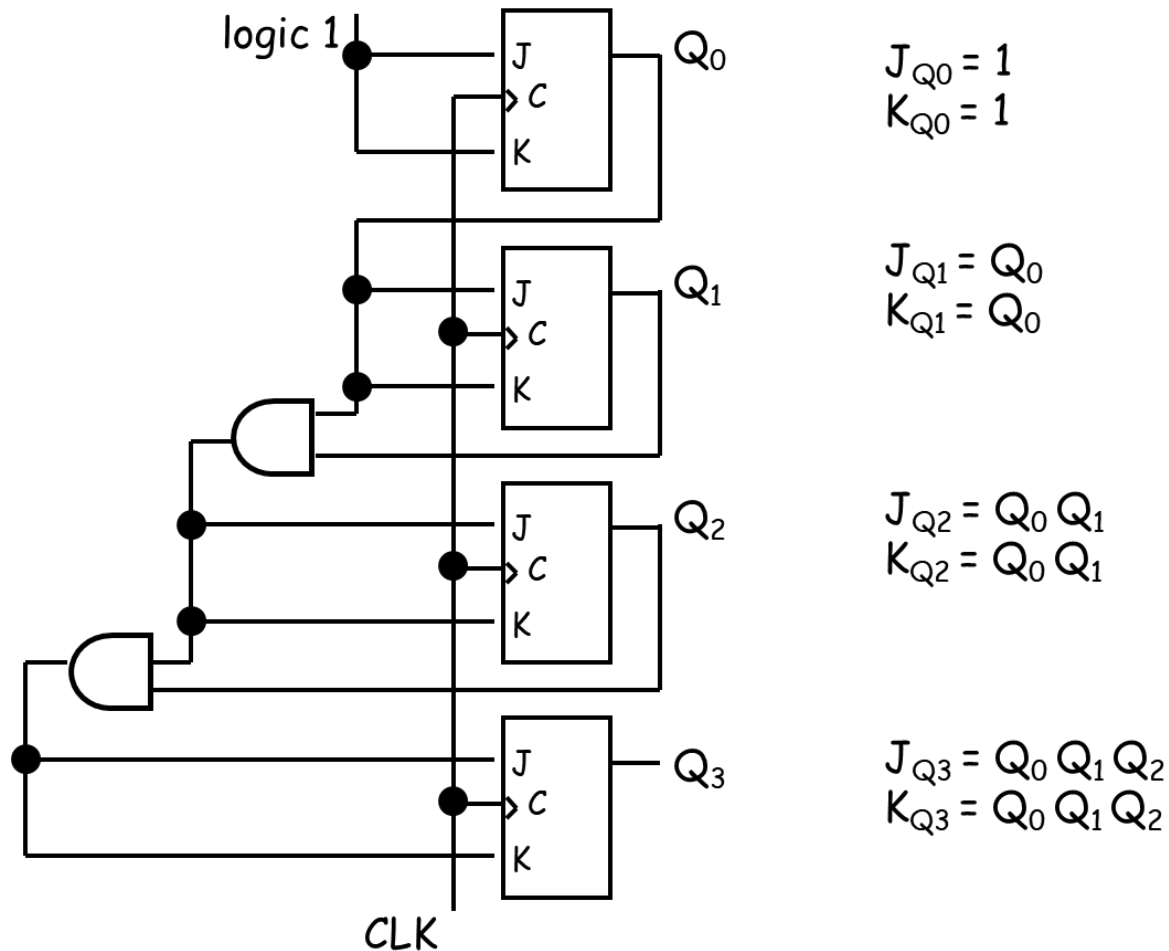
	1	X	X
	1	X	X
	1	X	X
	1	X	X

$$J_{Q1} = Q_0$$

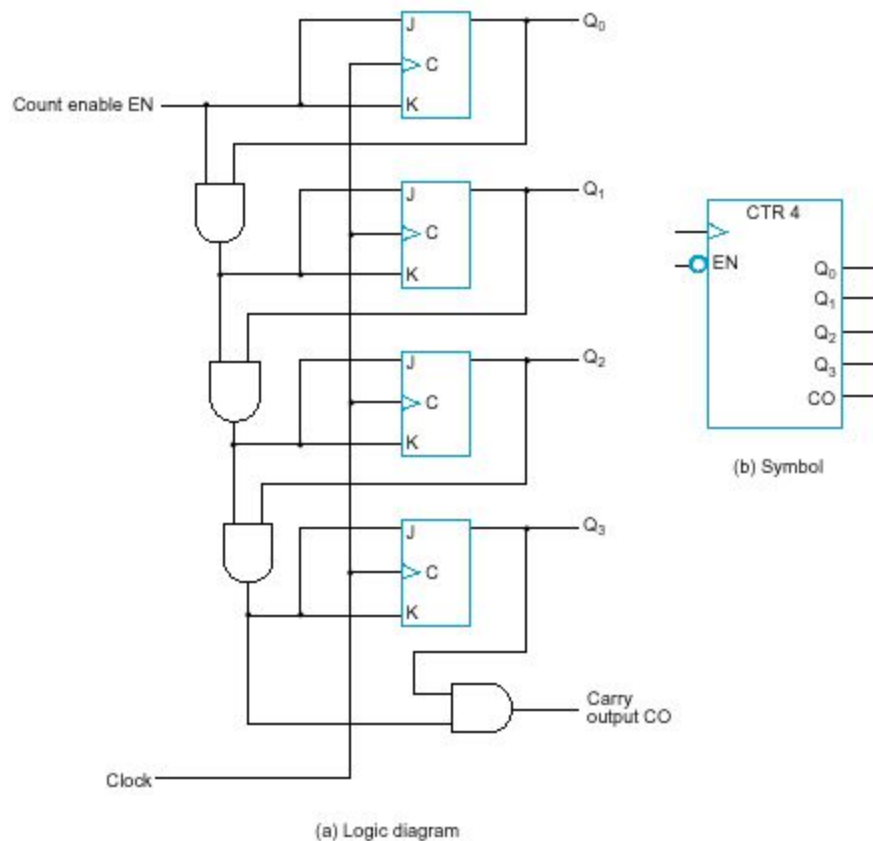
X	X	1	
X	X	1	
X	X	1	
X	X	1	

$$K_{Q1} = Q_0$$

JK synchronous binary counter



JK synchronous binary counter with EN and CO



EN = enable control signal, when 0 counter remains in the same state, when 1 it counts

$$\begin{aligned}
 J_{Q0} &= 1 \cdot EN \\
 K_{Q0} &= 1 \cdot EN \\
 J_{Q1} &= Q_0 \cdot EN \\
 K_{Q1} &= Q_0 \cdot EN \\
 J_{Q2} &= Q_0 Q_1 \cdot EN \\
 K_{Q2} &= Q_0 Q_1 \cdot EN \\
 J_{Q3} &= Q_0 Q_1 Q_2 \cdot EN \\
 K_{Q3} &= Q_0 Q_1 Q_2 \cdot EN \\
 CO &= Q_0 Q_1 Q_2 Q_3 \cdot EN
 \end{aligned}$$

CO = carry output signal:
used to extend the counter to
more stages

...how?

Serial vs parallel counters

- If serial gating (chain of gates, info ripples through) is used
☐ serial counter
- If serial gating is replaced with parallel gating (this is analogous with ripple-logic replaced with carry-lookahead logic in our adder designs)
☐ parallel counter
- Advantage of parallel over serial counter: faster in certain occasions (1111 ☐ 0000)

Up – Down binary counters



Up – Down binary counters

UD	Q2	Q1	Q0	Q2.D	Q1.D	Q0.D	UD	Q2	Q1	Q0	Q2.D	Q1.D	Q0.D
0	0	0	0	0	0	1	1	0	0	0	1	1	1
0	0	0	1	0	1	0	1	0	0	1	0	0	0
0	0	1	0	0	1	1	1	0	1	0	0	0	1
0	0	1	1	1	0	0	1	0	1	1	0	1	0
0	1	0	0	1	0	1	1	1	0	0	0	1	1
0	1	0	1	1	1	0	1	1	0	1	0	0	0
0	1	1	0	1	1	1	1	1	1	0	1	0	1
0	1	1	1	0	0	0	1	1	1	1	1	1	0

Up-Counter

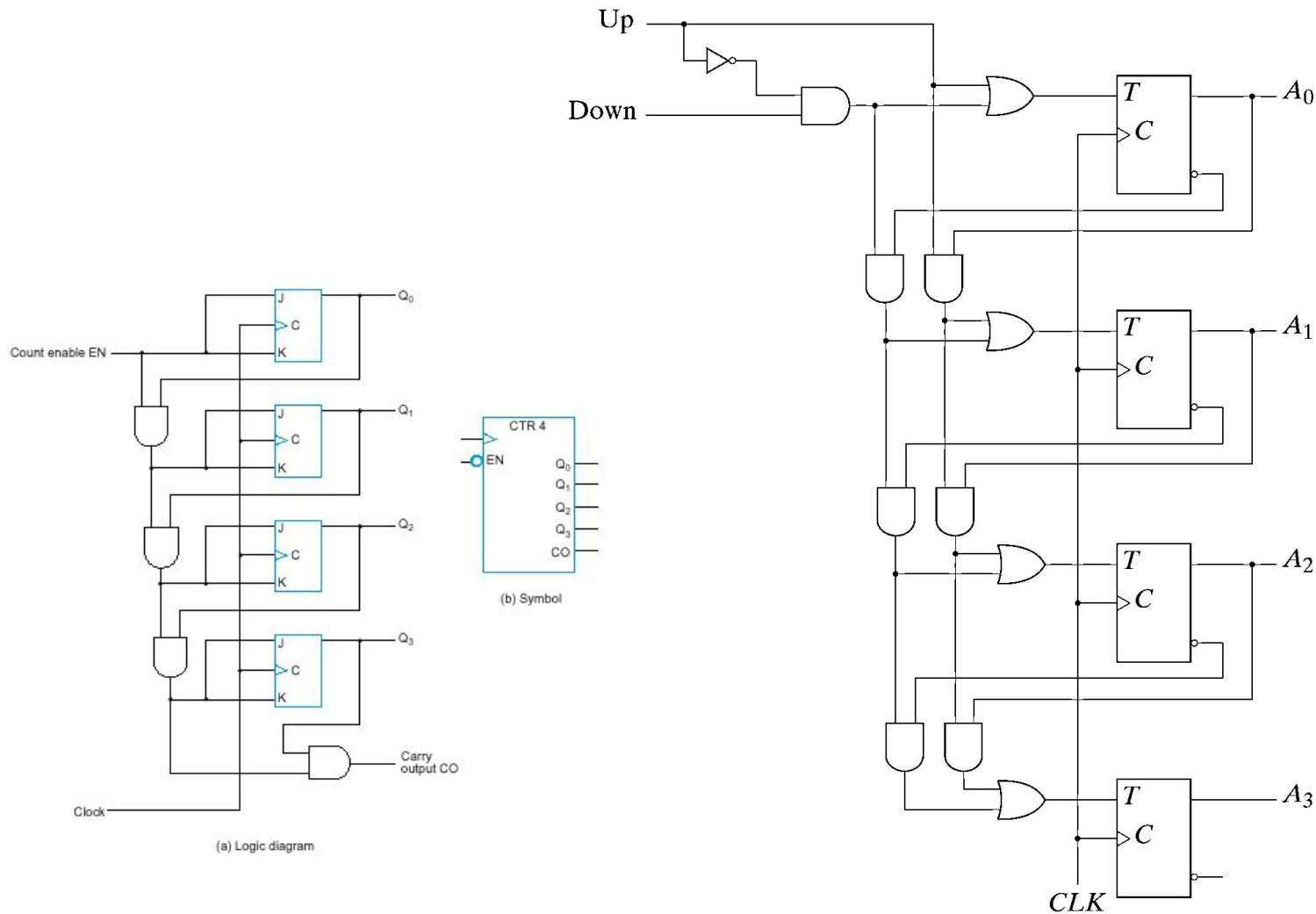
Down-Counter

Up – Down binary counters

UD Q2 \	00	01	11	10
00				
01				
11				
10				

Fill-in the Karnaugh maps for Q2.D, Q1.D, and Q0.D, simplify, and derive the logic diagram using
(a) D-FFs and (b) T-FFs

Up – Down binary counters

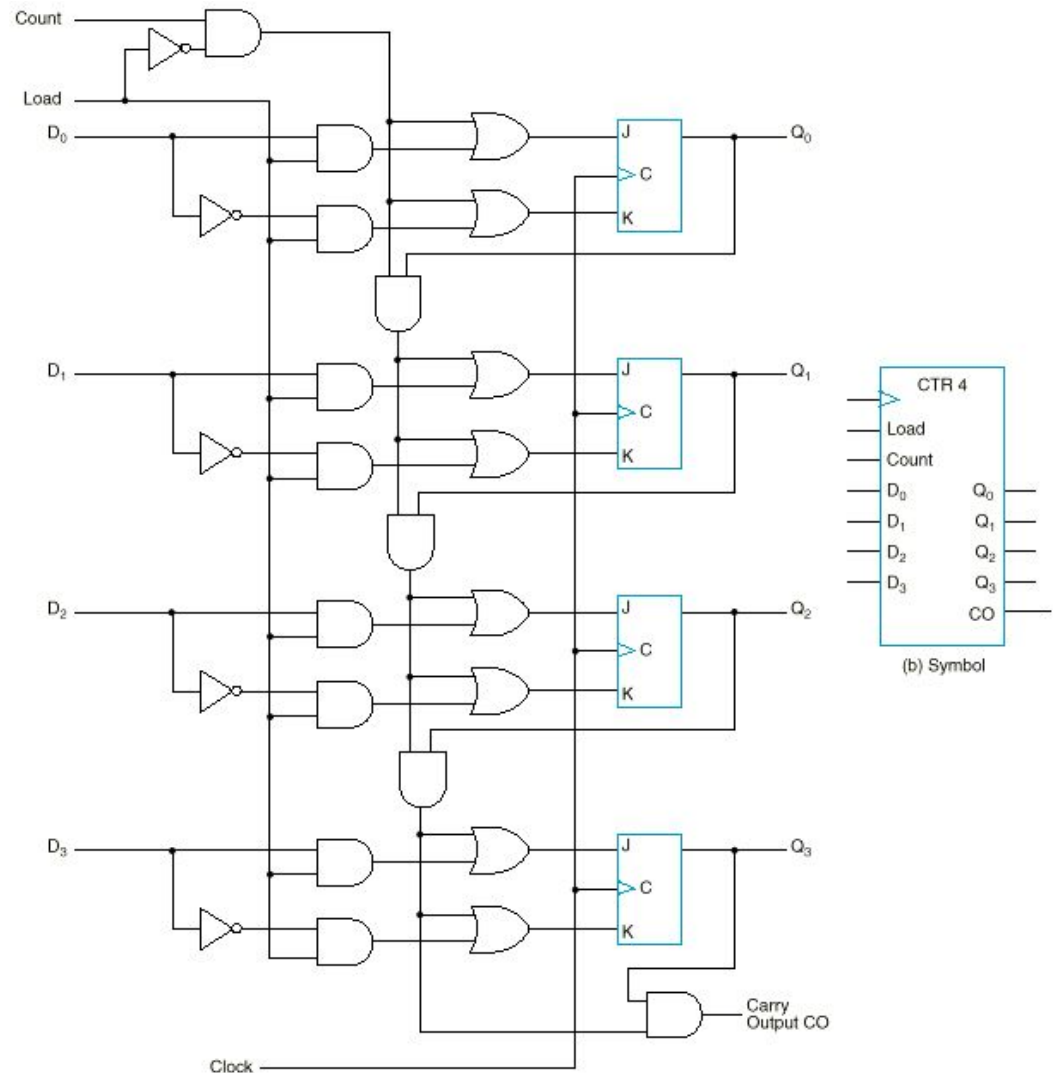


Binary counter with parallel load

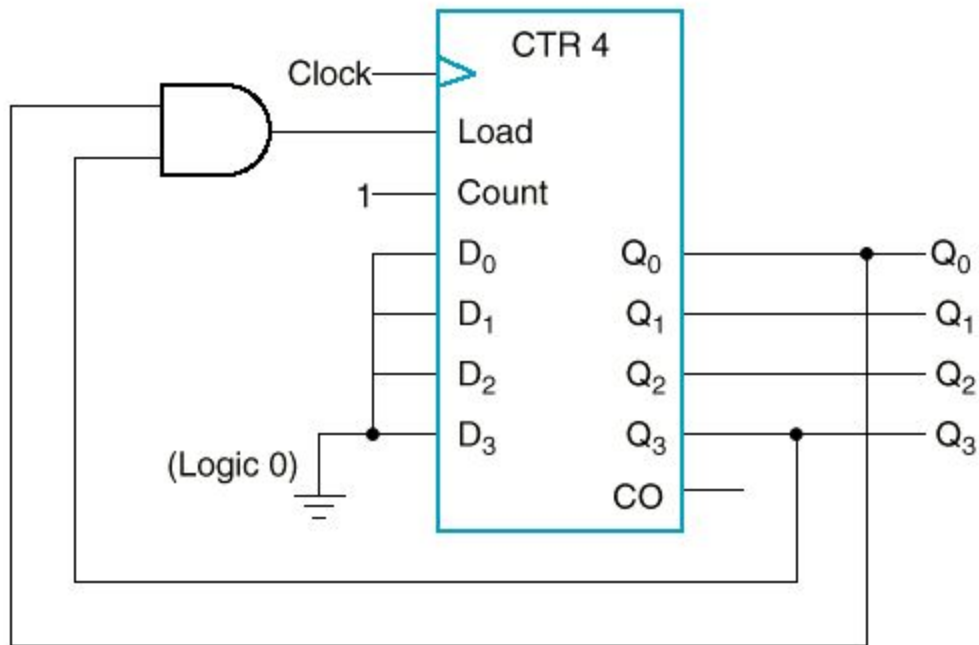


- Ability to set the counter to an arbitrary value to begin counting

Load	Count	Operation
0	0	Nothing
0	1	Count
1	x	Load



BCD counter using parallel load counter

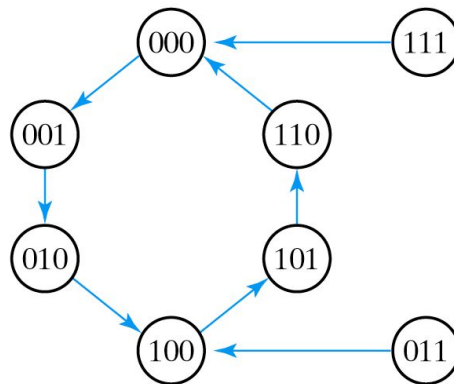
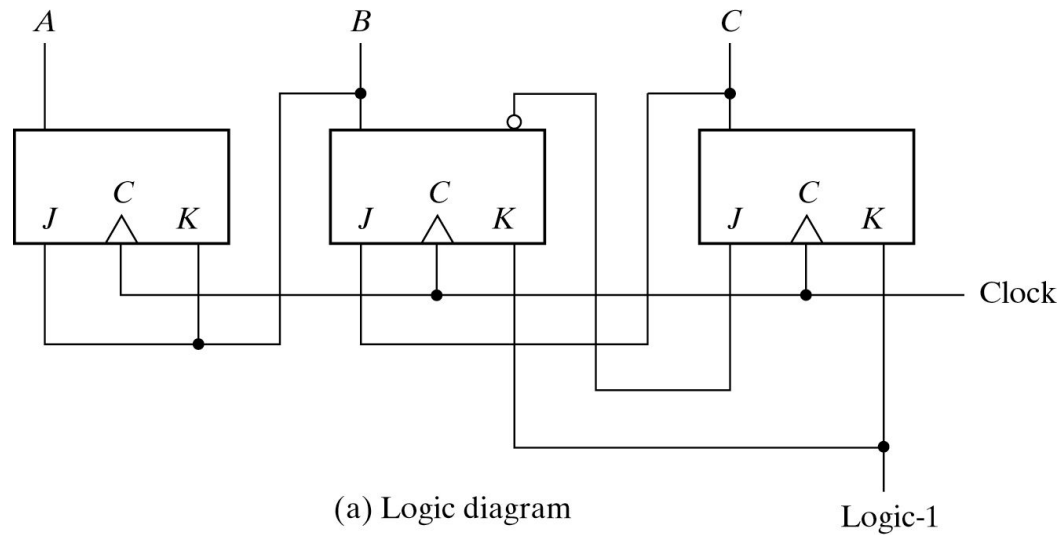


- The counter starts with an all-zero output.
- As long as the output of the AND gate is 0, each positive clock pulse transition increments the counter by one.
- When the output reaches the count of 1001, both Q_0 and Q_3 become 1, making the output of the AND gate equal to 1. This condition makes Load active, so on the next clock transition, the counter does not count, but is loaded from its four inputs.
- The value loaded then is 0000.

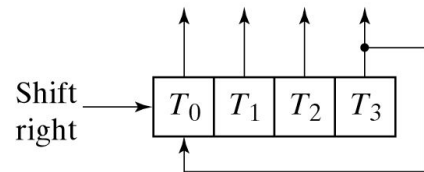
Arbitrary sequence counter

- Given an arbitrary sequence, design a counter that will generate the sequence
- Procedure:
 - Derive state table / diagram based on the sequence
 - Simplify
 - Build the circuit
- Example: Use D-FFs to design a sequence generator for the sequence: 0 7 6 1 0 (000 111 110 001 000)

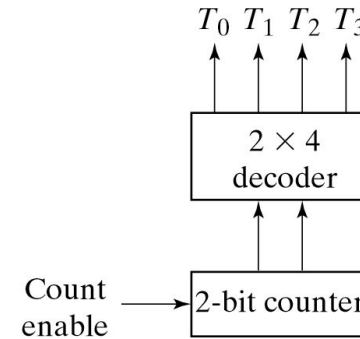
Counters with unused states



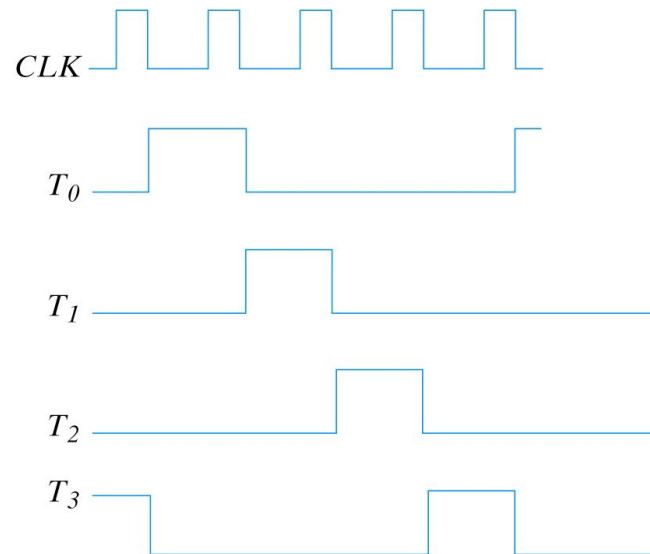
Ring counter



(a) Ring-counter (initial value = 1000)

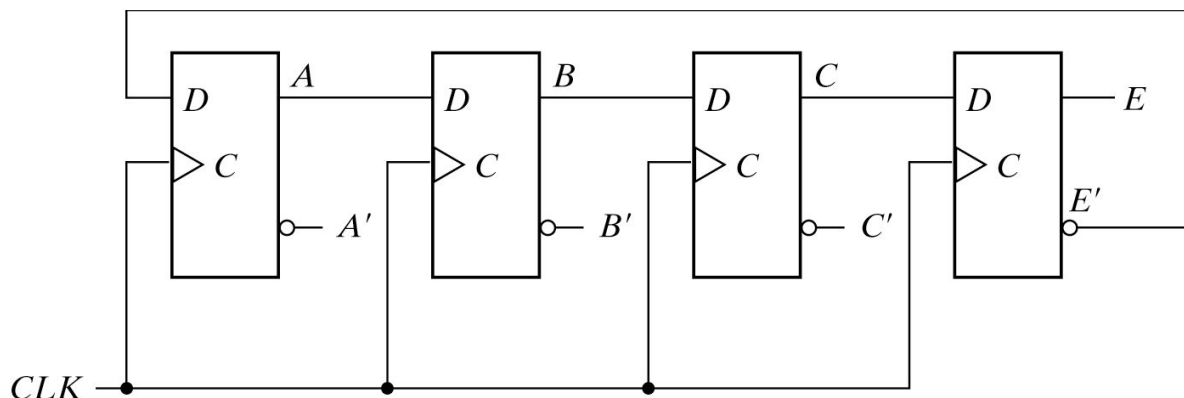


(b) Counter and decoder



(c) Sequence of four timing signals

Johnson counter



(a) Four-stage switch-tail ring counter

Sequence number	Flip-flop outputs				AND gate required for output
	A	B	C	E	
1	0	0	0	0	$A'E'$
2	1	0	0	0	AB'
3	1	1	0	0	BC'
4	1	1	1	0	CE'
5	1	1	1	1	AE
6	0	1	1	1	$A'B$
7	0	0	1	1	$B'C$
8	0	0	0	1	$C'E$

(b) Count sequence and required decoding